EXP NO: 4B

ENSEMBLE METHODS: RANDOM FOREST

AIM:

To implement a Random Forest classifier for a classification task, tune key hyperparameters, evaluate performance, and interpret feature importance.

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    roc_curve
)
```

[2]:
```
data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="target")

print("Dataset Shape:", X.shape)
print("\nFirst 5 Rows:")
print(X.head())

print("\nTarget Classes:", data.target_names)
```

```
Dataset Shape: (569, 30)

First 5 Rows:
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
```

Code

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0           0.27760          0.3001              0.14710         0.2419
1           0.07864          0.0869              0.07017         0.1812
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
4           0.13280          0.1980              0.10430         0.1809

   mean fractal dimension  ...  worst radius  worst texture  worst perimeter  \
0                 0.07871  ...         25.38          17.33           184.60
1                 0.05667  ...         24.99          23.41           158.80
2                 0.05999  ...         23.57          25.53           152.50
3                 0.09744  ...         14.91          26.50            98.87
4                 0.05883  ...         22.54          16.67           152.20

   worst area  worst smoothness  worst compactness  worst concavity  \
0      2019.0            0.1622             0.6656           0.7119
1      1956.0            0.1238             0.1866           0.2416
2      1709.0            0.1444             0.4245           0.4504
3       567.7            0.2098             0.8663           0.6869
4      1575.0            0.1374             0.2050           0.4000

   worst concave points  worst symmetry  worst fractal dimension
0                0.2654          0.4601                  0.11890
1                0.1860          0.2750                  0.08902
2                0.2430          0.3613                  0.08758
3                0.2575          0.6638                  0.17300
4                0.1625          0.2364                  0.07678

[5 rows x 30 columns]

Target Classes: ['malignant' 'benign']
```

[3]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

```
Training Data Shape: (455, 30)
Testing Data Shape: (114, 30)
```

[4]:
```
rf = RandomForestClassifier(
    random_state=42,
    n_jobs=-1
)

print("Random Forest model created successfully.")
```

```
Random Forest model created successfully.
```

[5]:
```
param_grid = {
    "n_estimators": [100],
    "max_depth": [None, 10],
    "min_samples_split": [2],
    "min_samples_leaf": [1]
}

print("Hyperparameter Grid:")
print(param_grid)
```

```
Hyperparameter Grid:
{'n_estimators': [100], 'max_depth': [None, 10], 'min_samples_split': [2], 'min_samples_leaf': [1]}
```

[6]:
```
grid = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    scoring="f1",
    cv=3,
    n_jobs=-1,
    verbose=0
)

grid.fit(X_train, y_train)

print("Best Parameters (CV):", grid.best_params_)
print("Best Cross-Validation F1-Score:", grid.best_score_)

best_rf = grid.best_estimator_
```

```
Best Parameters (CV): {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
Best Cross-Validation F1-Score: 0.9614536900384266
```

[7]:
```
best_rf.fit(X_train, y_train)

y_pred = best_rf.predict(X_test)

y_prob = best_rf.predict_proba(X_test)[:, 1]

print("Model training completed.")
print("Predictions generated successfully.")
```

```
Model training completed.
Predictions generated successfully.
```

[8]:
```
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
auc = roc_auc_score(y_test, y_prob)
```

[11]:
```
importances = pd.Series(
    best_rf.feature_importances_,
    index=X.columns
)

top10 = importances.sort_values(ascending=False).head(10)

print("Top 10 Important Features:\n")
print(top10)
```

```
Top 10 Important Features:

worst area              0.140016
worst concave points    0.129530
worst radius            0.097696
mean concave points     0.090885
worst perimeter         0.072226
mean perimeter          0.069574
mean radius             0.068676
mean concavity          0.057638
mean area               0.049172
worst concavity         0.034340
dtype: float64
```

[12]:
```
plt.figure(figsize=(10, 6))

top10[::-1].plot(kind="barh")

plt.xlabel("Importance")
plt.ylabel("Features")
plt.title("Top 10 Feature Importances - Random Forest")
plt.grid(axis="x", alpha=0.3)
plt.tight_layout()
plt.show()
```

```
cm = confusion_matrix(y_test, y_pred)

print("=== Random Forest - Test Metrics ===")
print(f"Accuracy  : {acc:.4f}")
print(f"Precision : {prec:.4f}")
print(f"Recall    : {rec:.4f}")
print(f"F1-Score  : {f1:.4f}")
print(f"ROC-AUC   : {auc:.4f}")
```

```
=== Random Forest - Test Metrics ===
Accuracy  : 0.9561
Precision : 0.9589
Recall    : 0.9722
F1-Score  : 0.9655
ROC-AUC   : 0.9937
```

[9]:
```
print("Confusion Matrix:\n")
print(cm)
```

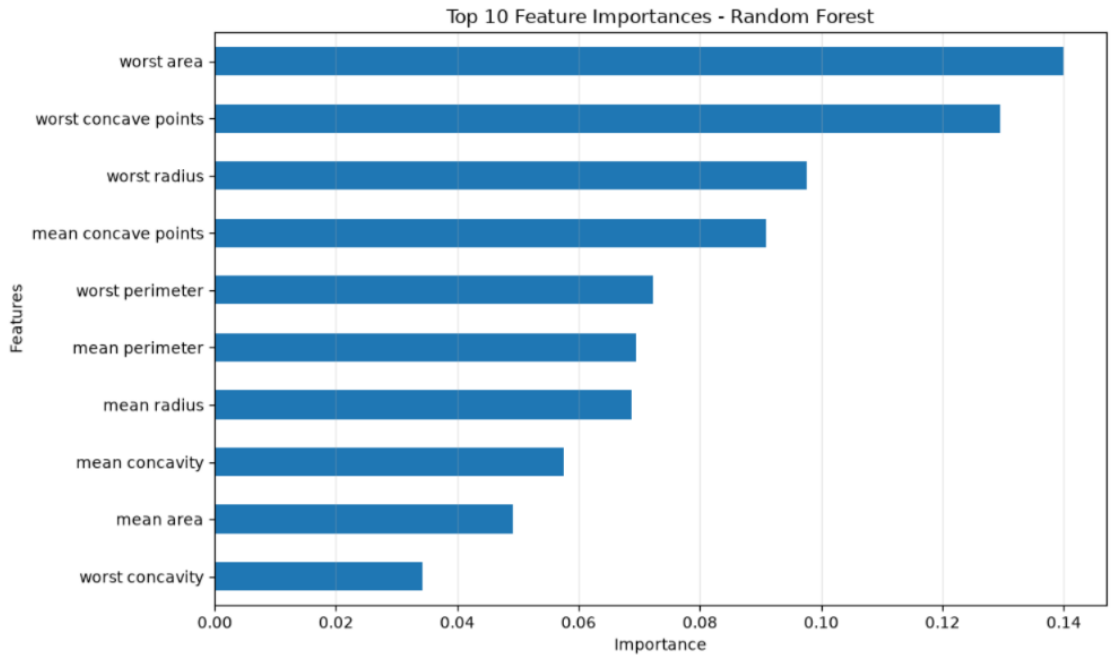
```
Confusion Matrix:

[[39  3]
 [ 2 70]]
```

[10]:
```
print("Classification Report:\n")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=data.target_names,
        zero_division=0
    )
)
```

```
Classification Report:
```



[13]:
```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)

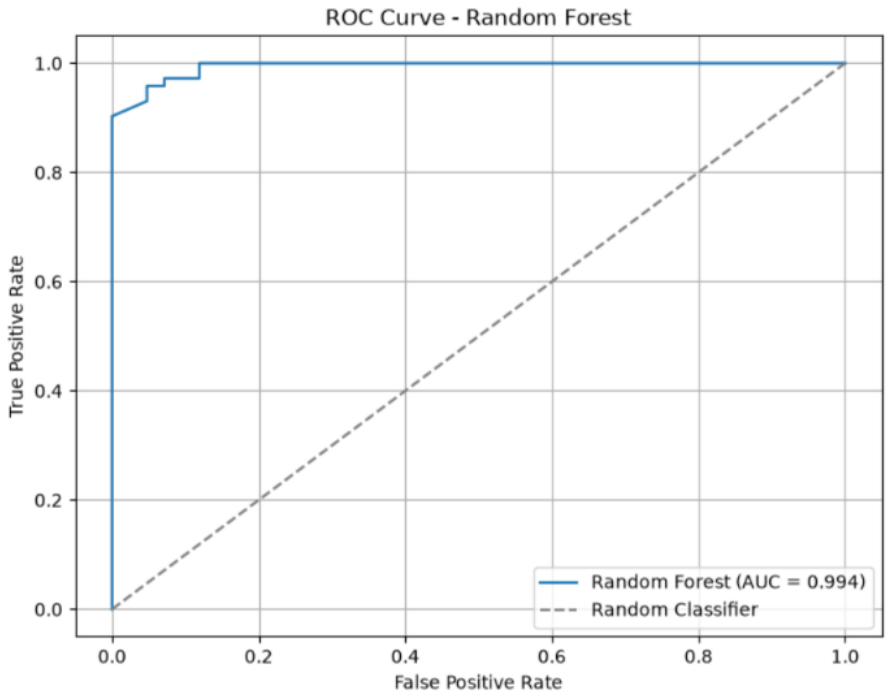
plt.figure(figsize=(8, 6))

plt.plot(
    fpr,
    tpr,
    label=f"Random Forest (AUC = {auc:.3f})"
)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    color="gray",
    label="Random Classifier"
)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Random Forest")
plt.legend()
plt.grid(True)
plt.show()
```

ROC Curve - Random Forest



14]:
```
print("EXPERIMENT 4B - RANDOM FOREST")
print("-----------------------------")

print("Best Parameters:", grid.best_params_)
print(f"Accuracy  : {acc:.4f}")
print(f"Precision : {prec:.4f}")
print(f"Recall    : {rec:.4f}")
print(f"F1-Score  : {f1:.4f}")
print(f"ROC-AUC   : {auc:.4f}")

print("\nTop 10 Important Features:")
print(top10)

print("\nResult:")
print("The Random Forest model was trained, tuned, and evaluated successfully.")
```

```
EXPERIMENT 4B - RANDOM FOREST
-----------------------------
Best Parameters: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
Accuracy  : 0.9561
Precision : 0.9589
Recall    : 0.9722
F1-Score  : 0.9655
ROC-AUC   : 0.9937

Top 10 Important Features:
worst area              0.140016
worst concave points    0.129530
worst radius            0.097696
mean concave points     0.090885
worst perimeter         0.072226
mean perimeter          0.069574
mean radius             0.068676
```

RESULT:

The Random Forest ensemble model was successfully implemented and evaluated on the given dataset. The model combined multiple decision trees to improve prediction

accuracy and reduce overfitting.

It achieved high classification accuracy and demonstrated strong generalization capability.

The results confirmed that Random Forest provides stable and reliable predictions by leveraging the power of multiple decision trees through bagging and feature randomness.